COMMUNITY OVER CODE · GLASGOW 2026

WireGuard for Apache NuttX

Bringing Secure Remote Access to Embedded Devices

Satoru Akita

Sony Semiconductor Solutions



SLIDES

About Me



Satoru Akita — edge AI engineer at Sony Semiconductor Solutions, working with Apache NuttX from the application side.



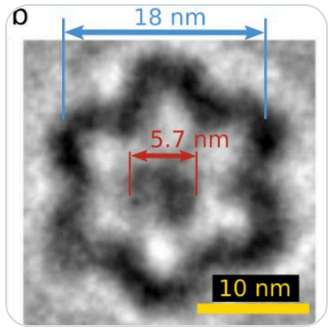
my icon on social media



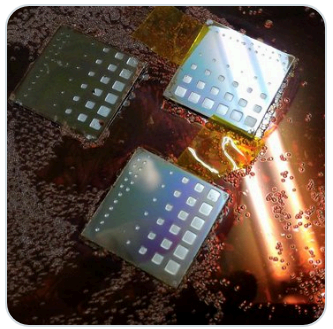
MY BACKGROUND & ACTIVITIES



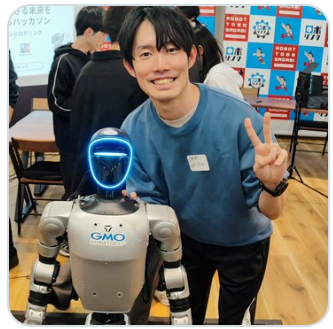
Rocket / Aerospace Engineering



DNA Nanorobotics



Semiconductor Research



Robots



Open-Source Conference Volunteer

Satoru Akita @fox_aki310ooooo

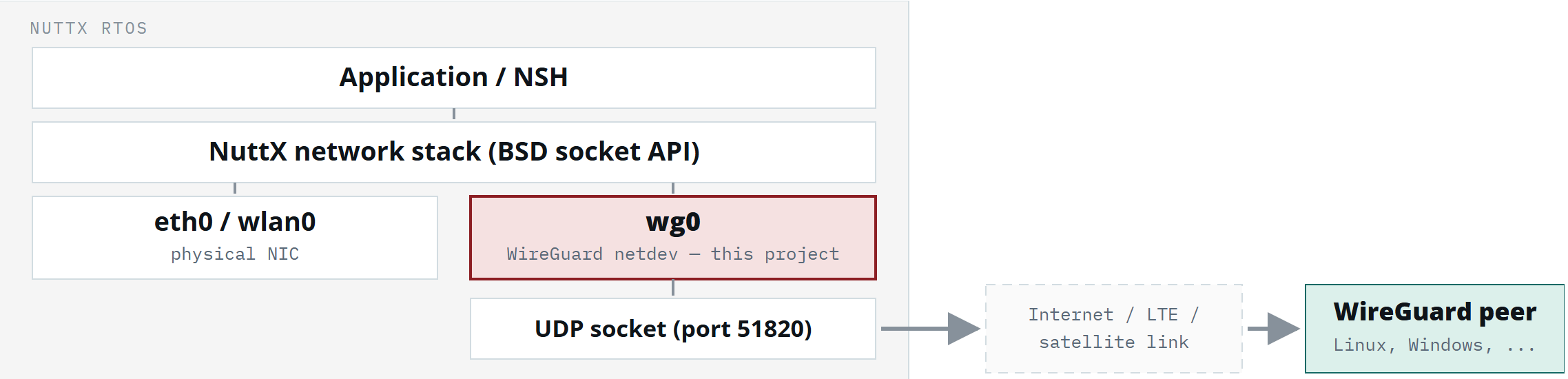
AGENDA

1. **Overview & background** — ~5 MIN
   Project origins · NuttX & Sony · my experience
2. **The Problem & Solution** — ~4 MIN
   Remote access challenge · why WireGuard
3. **The Development Plan** — ~2 MIN
   Reuse wireguard-lwip · adapt to NuttX
4. **Version 1: Application-Based Implementation** — ~4 MIN
   First working tunnel · debugging challenges
5. **Version 2: The Kernel Driver** — ~5 MIN
   Why a driver · integration challenges · validation
6. **Live Demo** — ~3 MIN
   Secure remote access · Stack-chan control
7. **Summary & Reflections** — ~4 MIN
   Achievements · personal reflections · community

AGENDA

1. **Overview & background** ~5 MIN
   Project origins · NuttX & Sony · my experience
2. **The Problem & Solution** ~4 MIN
   Remote access challenge · why WireGuard
3. **The Development Plan** ~2 MIN
   Reuse wireguard-lwip · adapt to NuttX
4. **Version 1: Application-Based Implementation** ~4 MIN
   First working tunnel · debugging challenges
5. **Version 2: The Kernel Driver** ~5 MIN
   Why a driver · integration challenges · validation
6. **Live Demo** ~3 MIN
   Secure remote access · Stack-chan control
7. **Summary & Reflections** ~4 MIN
   Achievements · personal reflections · community

PROJECT OVERVIEW



LIGHTWEIGHT VPN

Bring WireGuard to NuttX-based microcontrollers.

VIRTUAL INTERFACE

Use `wg0` as a network interface for the VPN tunnel.

SECURE COMMUNICATION

Exchange encrypted packets with WireGuard peers over UDP.

NuttX devices can now talk IP securely, like any other WireGuard peer.

How it started

MID FEB
Read the GSoC idea list.
Started to study.

EARLY MAR
Proposed the plan.

MAR 20
Alan told me about this conference. Submitted a CfP.

EARLY APR
Sent the GSoC proposal.

EARLY MAY
CfP accepted. GSoC proposal not selected.
Kept going anyway.

OCT 11–14
Glasgow — this talk.

FEB | MAR | APR | MAY | JUN | JUL | AUG | SEP | OCT

DEVELOPMENT

MAY – JUL
Studied NuttX and WireGuard, explored approaches

AUG
V1

SEP –
V2

VERSION 1
Application (FLAT) · hardware testing

→

VERSION 2
Kernel driver

The GSoC proposal was not selected, but the work went on.

A QUICK OVERVIEW OF APACHE NUTTX

Apache NuttX

A small RTOS for microcontrollers

- **POSIX-like real-time OS**
- **Small footprint**
- **Broad hardware support**
  15+ CPU architectures · 300+ boards
- **Apache open-source project**

RTOS	STANDARD APIS
FreeRTOS	Its own task API
Zephyr	Own API, optional POSIX layer
NuttX	**POSIX-like by design**

USED IN REAL DEVICES



Sony
Audio products, SPRESENSE



Fitbit
Fitness trackers (Charge 3 / 4)



PX4
Drone flight controllers



SORA-Q
Lunar robot on Japan's 2024 Moon mission

1,000+

Xiaomi OpenVela
Reported products: watches, speakers, displays

Apache NuttX is a POSIX-like RTOS used in a wide range of real-world embedded devices.

Sources: nuttx.apache.org/market-share · "NuttX at Fitbit" (2020) · docs.px4.io · Sony Developer Blog. Images: Sony; Wikimedia Commons (Charge 3: Vemerce, CC BY-SA 4.0; Pixhawk: Pixhawk project, CC BY 4.0; SORA-Q: テレストレラッソ, CC BY-SA 4.0)

Sony's history with NuttX

2013–15 — **Early adoption**
Ported NuttX to LC823425 / LC823450; first commercial audio products shipped in 2015.

2016–17 — **Commercial products**
Walkman NW-WS620, SMR-10, ICD-TX800 and ICD-PX470 used NuttX.

2018 — **Xperia Ear Duo & SPRESENSE**
Xperia Ear Duo shipped with CXD5602 / NuttX; SPRESENSE launched as a multicore development board.

2019–20 — **Upstream collaboration**
Sony engineers upstreamed CXD56xx platform support and drivers, and shared the work in public NuttX talks.

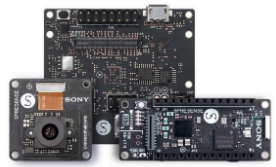
2020s — **Edge sensing**
Expanded camera, GNSS, connectivity and edge-sensing capabilities.



Walkman NW-WS620



Xperia Ear Duo



SPRESENSE

Sony's NuttX journey began with audio products, expanded through SPRESENSE, and evolved into long-term open-source collaboration.

Sources: M. Ishikawa, Sony NuttX talks (2018–19) · Sony, "Running NuttX on Spresense" (2019) · A. Jerpelea (2020) · Apache NuttX CXD56xx docs

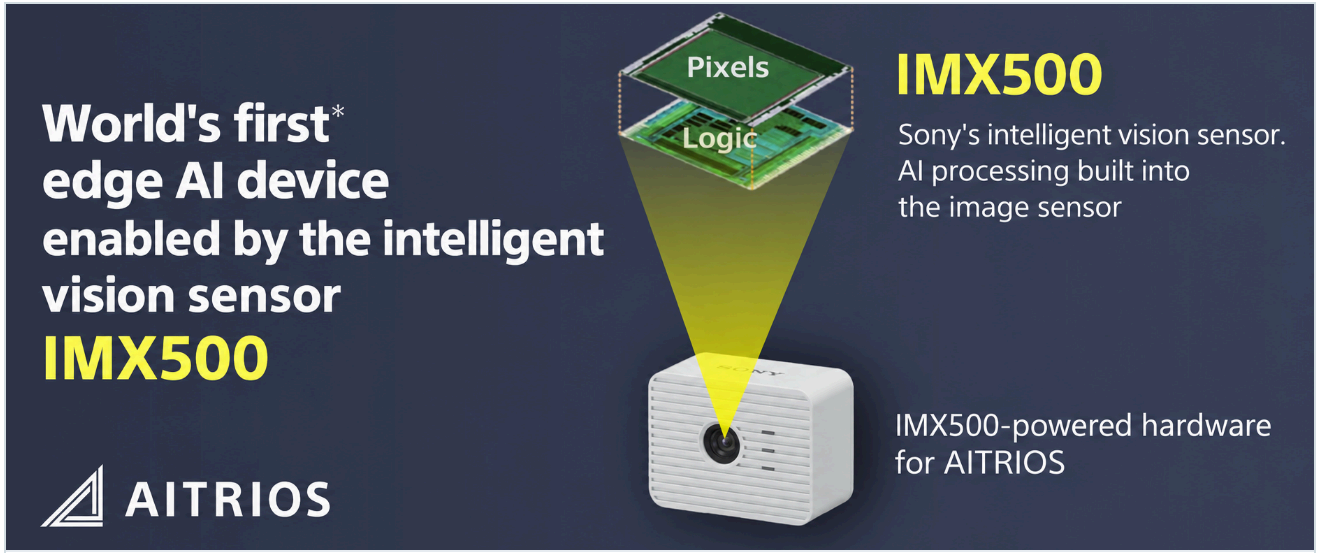
My Experience with NuttX



SPRESENSE — Satellite Project

Sony's low-power board, the on-board computer of the ARICA-2 CubeSat (Aoyama Gakuin University, launched April 2026).

I took part through an internal, grassroots engineering initiative.



AITRIOS — Edge AI Sensing Platform

Sony's platform for managing edge AI cameras and running inference on them.

I worked on setting up and operating camera systems that use ESP32 and Apache NuttX.

I wanted a secure way to access and troubleshoot remote embedded devices.

Images: ARICA project (ARICA CubeSat) · Sony Semiconductor Solutions (AITRIOS). ARICA-2: nanosats.eu/sat/arica-2 · SmallSat 2026 paper (Mabuchi et al.)

AGENDA

1 **Overview & background** ~5 MIN
Project origins · NuttX & Sony · my experience

2 **The Problem & Solution** ~4 MIN
Remote access challenge · why WireGuard

3 **The Development Plan** ~2 MIN
Reuse wireguard-lwip · adapt to NuttX

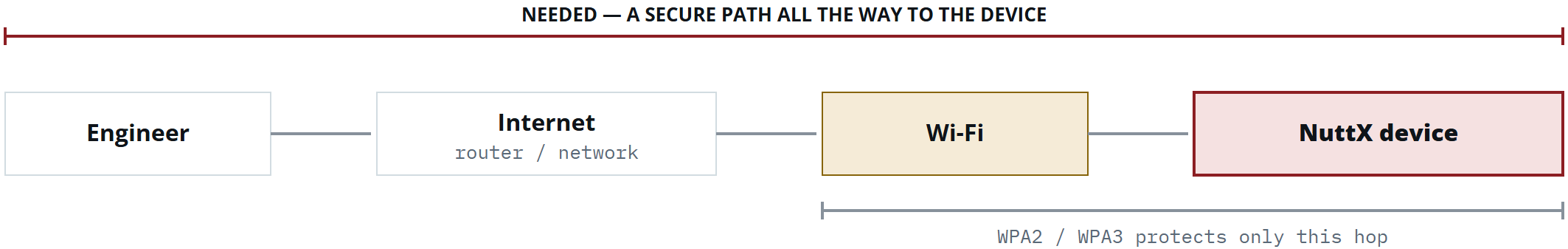
4 **Version 1: Application-Based Implementation** ~4 MIN
First working tunnel · debugging challenges

5 **Version 2: The Kernel Driver** ~5 MIN
Why a driver · integration challenges · validation

6 **Live Demo** ~3 MIN
Secure remote access · Stack-chan control

7 **Summary & Reflections** ~4 MIN
Achievements · personal reflections · community

Problem: How Can We Securely Access Remote Devices?



DEVICES ARE FAR AWAY

Embedded devices sit at remote sites and on other networks.

REMOTE DIAGNOSIS

Engineers need to check and fix them when something goes wrong.

A SECURE PATH

Wi-Fi security protects the radio link. Remote access needs its own secure path.

We needed a secure way to access and troubleshoot remote NuttX devices.

Options: Ways to Reach a Remote Device

WAY	ANY IP SERVICE (TELNET, WEB…)?	NO SERVER IN THE MIDDLE?	PROTECTED UP TO
Open a port to the Internet	yes	yes	the application itself
HTTPS / MQTT via a cloud	no, only your API or messages	no, a server or broker	the server or broker
Reverse tunnel / relay (SSH, WebSocket)	one port per tunnel	no, a relay host	the relay host
VPN on the router	yes, the whole LAN	yes	the router, not the LAN behind it
Carrier closed network	yes	no, the carrier	the carrier gateway
VPN on the device (this talk)	yes, any IP service	yes, any WireGuard peer	the device itself

> **Every option has trade-offs. I chose a VPN on the device, so the device itself is the end of the tunnel.**

Each option can be made secure. The last column shows where its protection ends. Mesh VPN agents (Tailscale, ZeroTier) are another “VPN on the device” option, but they usually need a bigger OS.

Solution: A VPN on the Device

Engineer · Linux · Windows
real, existing WireGuard peers

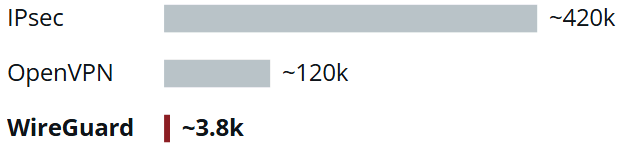
WIREGUARD TUNNEL

NuttX device
wg0

WHY WIREGUARD?

Small & simple

Far less code than IPsec or OpenVPN, so it is easier to review and to port.



Lines of code, 2018 — not today's counts

Modern cryptography

ChaCha20-Poly1305 and Curve25519, in one fixed, well-defined suite.

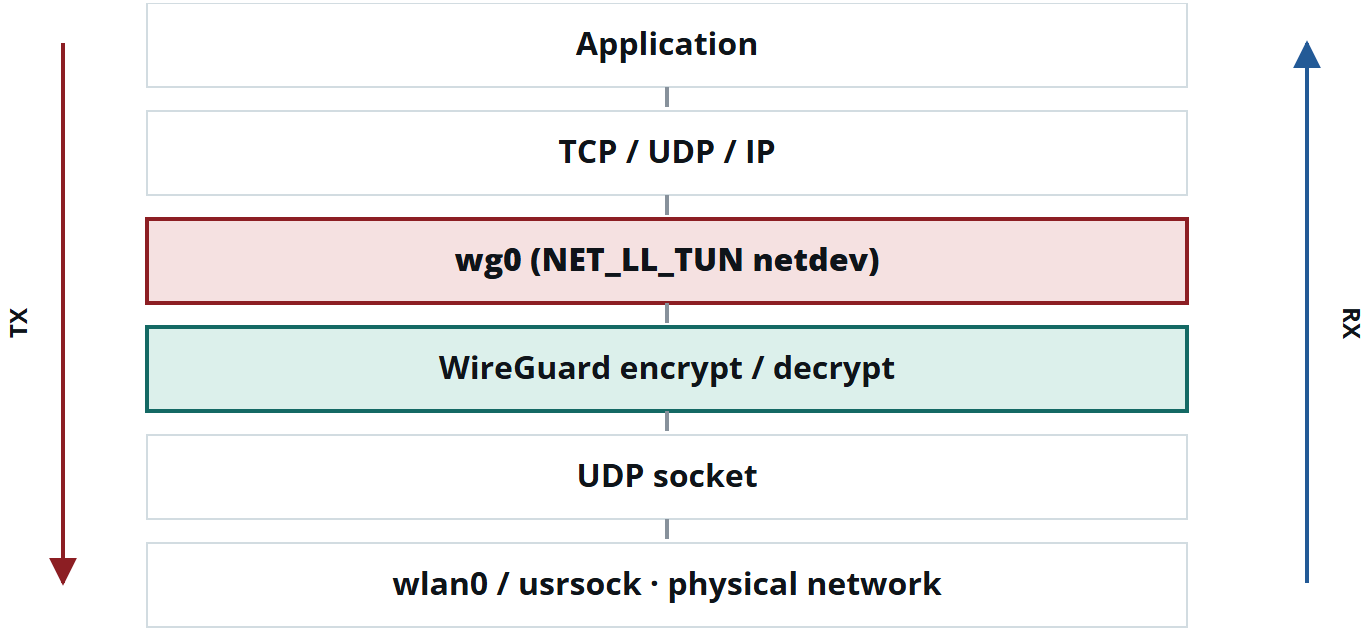
UDP-based tunnel

Encrypted IP packets over UDP. It works with existing WireGuard peers such as Linux, and the microcontroller itself is the VPN endpoint.

Run a lightweight, standard VPN directly on the NuttX device.

Lines of code (rounded): J. Donenfeld, "WireGuard", Linux Plumbers Conference 2018, wireguard.com/talks/lpc2018-wireguard-slides.pdf. Specific implementations, counted in 2018; the talk's own slides differ slightly for OpenVPN.

Architecture: where wg0 sits



To NuttX, wg0 is a normal network device. Under it, WireGuard turns its packets into encrypted UDP.

AGENDA

1 **Overview & background** ~5 MIN
Project origins · NuttX & Sony · my experience

2 **The Problem & Solution** ~4 MIN
Remote access challenge · why WireGuard

3 **The Development Plan** ~2 MIN
Reuse wireguard-lwip · adapt to NuttX

4 **Version 1: Application-Based Implementation** ~4 MIN
First working tunnel · debugging challenges

5 **Version 2: The Kernel Driver** ~5 MIN
Why a driver · integration challenges · validation

6 **Live Demo** ~3 MIN
Secure remote access · Stack-chan control

7 **Summary & Reflections** ~4 MIN
Achievements · personal reflections · community

Porting Strategy: Start from wireguard-lwip

STARTING POINT **github.com/smartalock/wireguard-lwip** — An existing open-source WireGuard for lwIP

KEEP

PROTOCOL & CRYPTO

Reuse the WireGuard protocol and cryptography, with two small fixes.

ADAPT

PLATFORM HOOKS

Implement NuttX-specific system functions.

REPLACE

LWIP NETWORK GLUE

Use NuttX networking APIs instead of lwIP.

We ported wireguard-lwip to NuttX by reusing the core, adapting platform hooks, and replacing the lwIP integration.

Mapping lwIP Networking to NuttX

CONCEPT	LWIP	NUTTX
Network device	struct netif	**struct net_driver_s**
Packet buffers	pbuf	**iob (d_iob)**
Registration	netif_add()	**netdev_register()**
Sending	output callback	**d_txavail → devif_poll()**
UDP “wire”	udp_pcb, udp_sendto()	**UDP socket, psock_sendto()**

> These are matches of ideas, not a one-to-one swap of APIs.

Porting meant using NuttX's own building blocks, not bringing lwIP into NuttX.

AGENDA

1 **Overview & background** ~5 MIN
Project origins · NuttX & Sony · my experience

2 **The Problem & Solution** ~4 MIN
Remote access challenge · why WireGuard

3 **The Development Plan** ~2 MIN
Reuse wireguard-lwip · adapt to NuttX

4 **Version 1: Application-Based Implementation** ~4 MIN
First working tunnel · debugging challenges

5 **Version 2: The Kernel Driver** ~5 MIN
Why a driver · integration challenges · validation

6 **Live Demo** ~3 MIN
Secure remote access · Stack-chan control

7 **Summary & Reflections** ~4 MIN
Achievements · personal reflections · community

Version 1: WireGuard as a NuttX application

GOAL

Run a standard WireGuard tunnel as a NuttX application.

Reuse the existing core and implement the NuttX integration.

LAYER	WORK NEEDED	IN THIS PORT
Protocol & crypto	**Almost none** portable C	Crypto byte-identical. **Two small fixes** to the protocol core. ~3,100 lines reused · ~40 changed
OS hooks	**Four functions** small	clock, `/dev/urandom`, TAI64N, load check ~120 new lines
Network glue	**The real work**	**netdev** + **UDP socket** + **RX task** ~1,200 new lines

Version 1 proved that WireGuard could run on NuttX.
It also exposed OS and networking problems that the first tunnel did not reveal.

Lines include comments, from git history. Reused: the vendored wireguard-lwip files. New: the first working tunnel, 2026-08-13. Fixes: 2026-09-18.

Where it was hard: six bugs

#	BUG	SHOWED UP IN	LESSON
1	The receive thread stopped when the `wg` command exited	apps version	`pthread_detach()` frees memory later. It does not keep a thread alive.
2	**ping works, TCP dies** `in detail`	**apps version**	**A file descriptor belongs to one task group.**
3	**“No real impact” was 10×** `in detail`	**apps version**	**Measure a quick fix before you call it free.**
4	Stack 94% full; saving the config froze the board	real boards only, never sim	Measure on real hardware. Sim's big stacks hide it.
5	Handshake OK, data fails: NuttX's own ChaCha20-Poly1305 `part 5`	kernel driver	A passing handshake does not prove the data path.
6	A 6 KB array on a 3 KB kernel stack `part 5`	BUILD_KERNEL (QEMU)	The same stack lesson, one level deeper.

Most of them hid behind a test that passed, so I first looked in the wrong place.

Bug ②: Ping Works. TCP Dies.

SYMPTOM

`ping` works: **0% loss.**

A TCP handshake even **completes**, then the connection dies.

```
psock_sendto: ret=-1 errno=9
EBADF — "Bad file descriptor"
```

ROOT CAUSE & FIX

Sends from the `wg_rx` task worked. Sends from the system work queue, which flushes TCP data, failed.

Cause: a file descriptor belongs to one **task group**. It is not valid in another one.

Fix: use `psock_*()` on a `struct socket`. No fd, so any task can use it.

If only some tasks fail, look at what each task owns. Here, it was the file descriptor.

Bug ③: “No Real Impact” Was 10× Slower

QUICK FIX

Check every 50 ms

Dropping fds also broke `poll()`, so I polled the socket. I thought the only cost was waking an idle CPU.

Measured: about 10× slower.

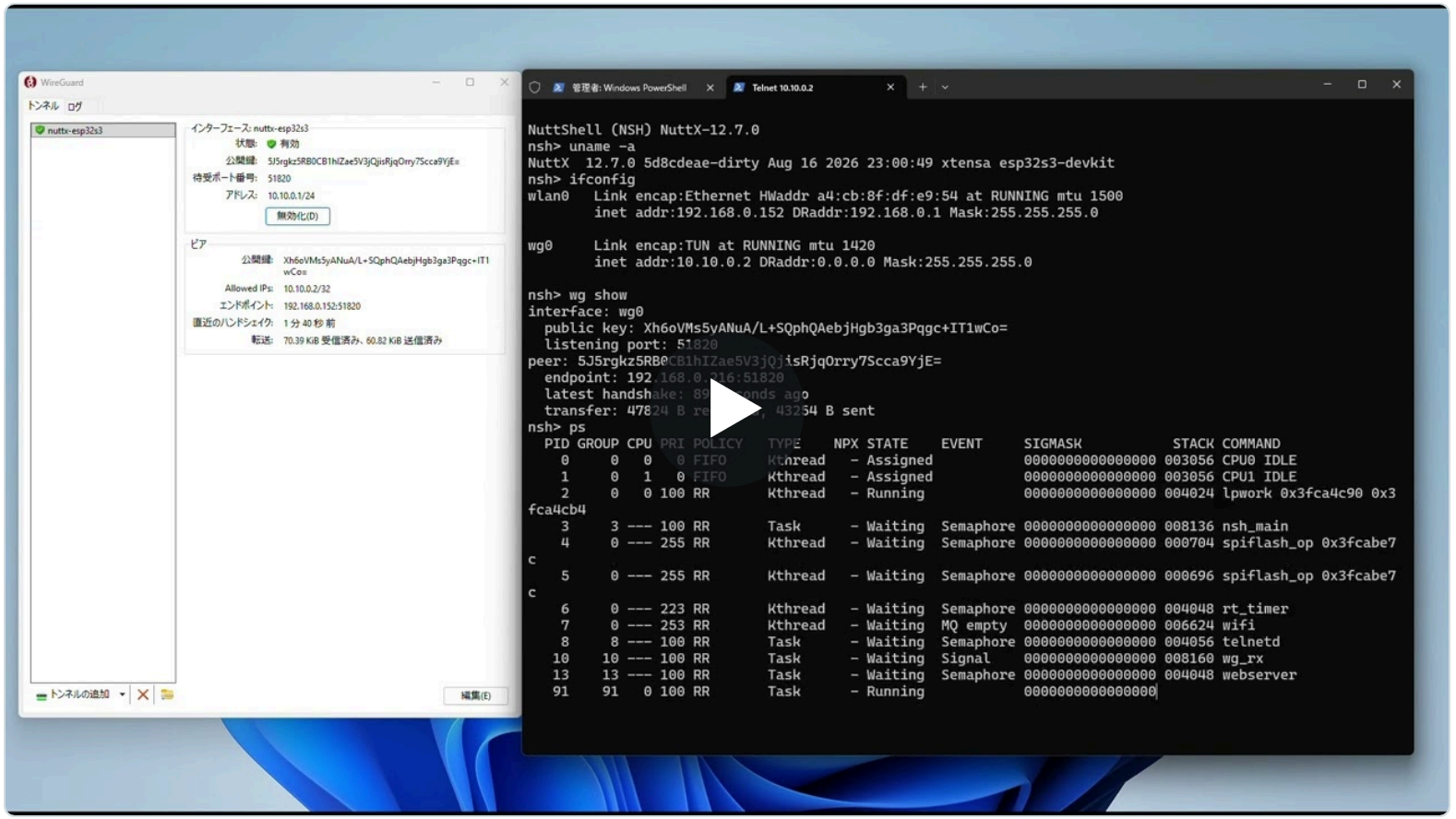
REAL FIX

Event-driven wake-up

A callback on `psock_poll()` and a semaphore: a real blocking wait. Each packet wakes the thread when it arrives.

A quick fix you never measure can look free. Measure it.

Version 1 demo: the first video




```
1  telnet 10.10.0.2
2  uname -a, ifconfig
3  webserver &
4  browser:
http://10.10.0.2/
```

ESP32-S3 DevKit, apps version (FLAT). USB unplugged.

Other end: the official Windows WireGuard client.

Video: youtu.be/1kyX2av5WG4

Version 1 was enough for a demo. Next, I made it a real NuttX driver.

AGENDA

1 **Overview & background** ~5 MIN
Project origins · NuttX & Sony · my experience

2 **The Problem & Solution** ~4 MIN
Remote access challenge · why WireGuard

3 **The Development Plan** ~2 MIN
Reuse wireguard-lwip · adapt to NuttX

4 **Version 1: Application-Based Implementation** ~4 MIN
First working tunnel · debugging challenges

5 **Version 2: The Kernel Driver** ~5 MIN
Why a driver · integration challenges · validation

6 **Live Demo** ~3 MIN
Secure remote access · Stack-chan control

7 **Summary & Reflections** ~4 MIN
Achievements · personal reflections · community

Version 2: WireGuard as a NuttX kernel driver

BEFORE — VERSION 1

An application

Lives in `apps/` and reaches into kernel internals (`psock_*`).

Works only in a **FLAT** build, and does not fit as an in-tree driver.

AFTER — VERSION 2

A kernel driver

`wg0` lives in `drivers/net/wireguard/`. A small `wg` command sets it up through **ioctl**.

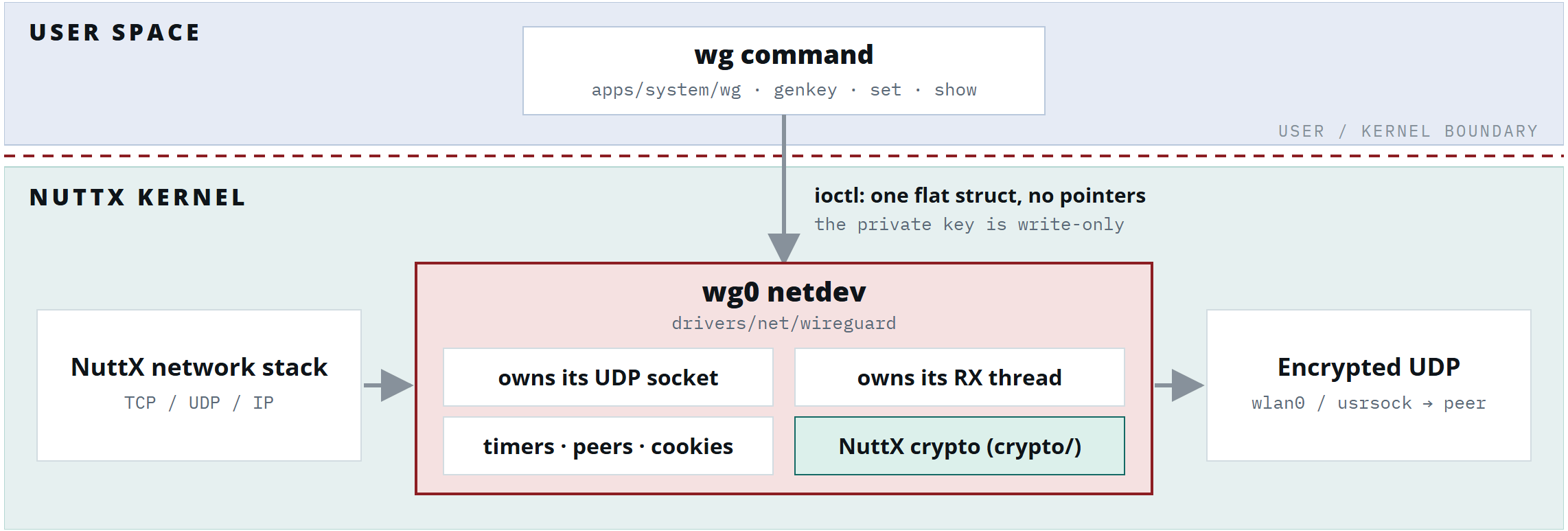
Made for **FLAT, PROTECTED and KERNEL** builds.

BUILD MODES

FLAT	Applications and kernel share one address space.
PROTECTED	MPU separates application and kernel memory.
KERNEL	MMU provides separate address spaces.

The goal: a proper kernel driver for every NuttX build mode, in a form that is easier to review upstream.

The Kernel Design: The Big Picture



The driver owns everything that must keep running. The command only sends settings, and then it can exit.

Two Issues Found During Kernel Integration

CRYPTOGRAPHY — ChaCha20-Poly1305

NUTTX `crypto/chachapoly.c`

- The nonce counter sits in different bytes from the RFC 8439 / WireGuard layout.
- Counter 0 still matches: **the handshake works, but data packets after the first fail.**
- A two-line fix with test vectors, kept as its own change.

KERNEL STACK — Stack Overflow

THIS DRIVER, KERNEL BUILD ONLY

- Setting the peers put a **6 KB array** on a **3 KB kernel stack**: overflow, then panic.
- The sim (FLAT, big stacks) never showed it.
- A bug in the driver itself. The array now lives on the heap.

Porting WireGuard also surfaced findings worth sharing with the NuttX community.

Testing: What Ran Where

BUILD MODE	ENVIRONMENT	RESULT
FLAT	ESP32-S3 · SPRESENSE real boards	✓ **Verified**
KERNEL	QEMU rv-virt tunnel to Linux; 4 CPUs under load	✓ **Verified**
PROTECTED	QEMU rv-virt one fault found and fixed: 20/20 runs	✓ **Verified**
KERNEL / PROTECTED	Real hardware	— **Not tested yet**

✓ means it passed in the conditions noted here. It is not a guarantee for other hardware.

Verified on real boards in FLAT builds and on QEMU in KERNEL / PROTECTED builds.
Real-hardware testing for those modes remains open.

AGENDA

1 **Overview & background** ~5 MIN
Project origins · NuttX & Sony · my experience

2 **The Problem & Solution** ~4 MIN
Remote access challenge · why WireGuard

3 **The Development Plan** ~2 MIN
Reuse wireguard-lwip · adapt to NuttX

4 **Version 1: Application-Based Implementation** ~4 MIN
First working tunnel · debugging challenges

5 **Version 2: The Kernel Driver** ~5 MIN
Why a driver · integration challenges · validation

6 **Live Demo** ~3 MIN
Secure remote access · Stack-chan control

7 **Summary & Reflections** ~4 MIN
Achievements · personal reflections · community

Live Demo: Stack-chan over WireGuard

NETWORK SETUP



LIVE DEMO STEPS

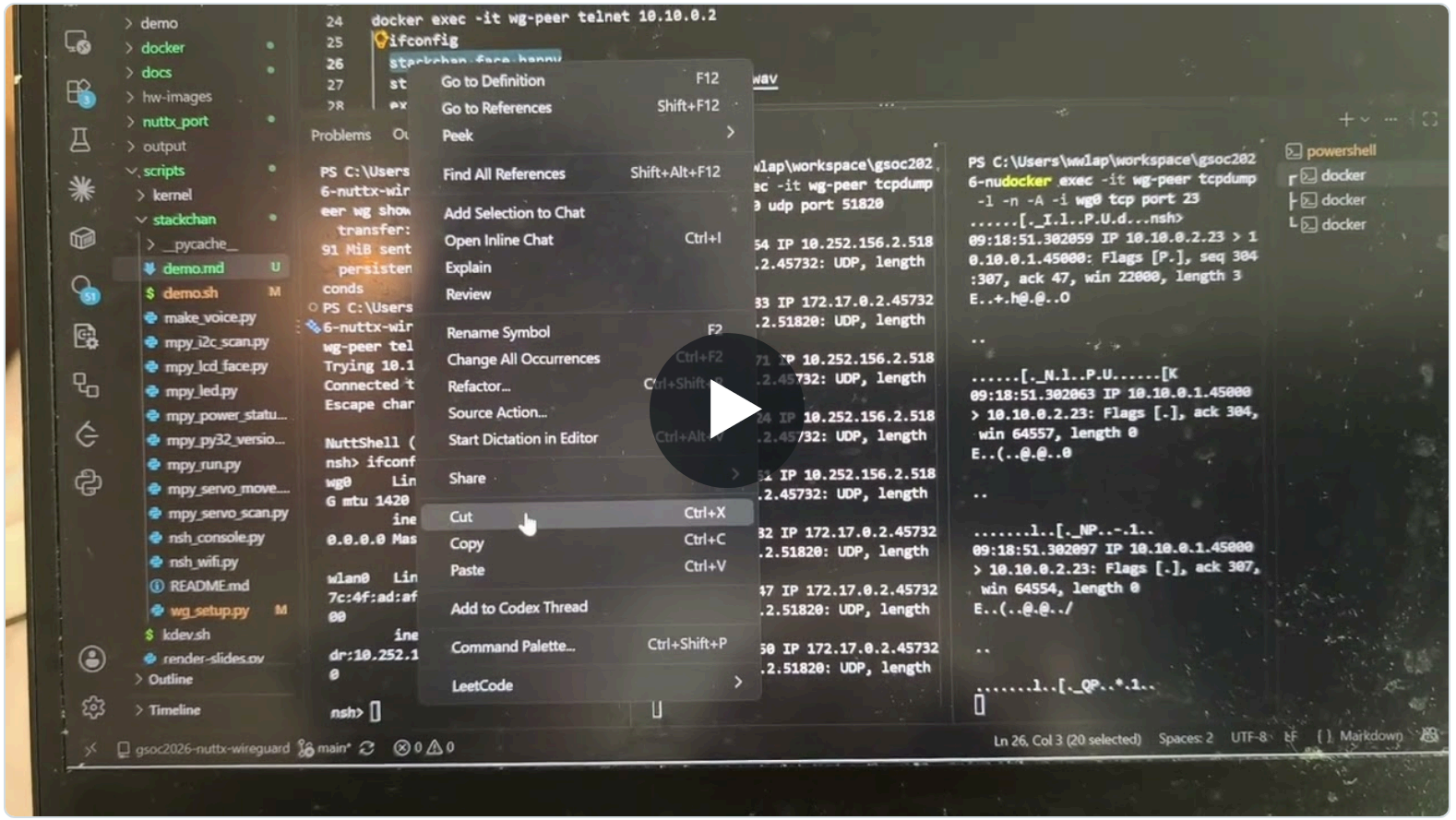
STEP	WHAT TO DEMONSTRATE	COMMAND / ACTION
1	**VPN Connection**	`wg show`
2	**Remote Login**	`telnet 10.10.0.2`
3	**Network Interface**	`ifconfig` → `wg0`
4	**Control Stack-chan**	Change face / Play voice



Stack-chan
Open-source palm-sized robot
github.com/meganetaaan/stack-chan

Photo: Stack-chan project (docs/images, Apache License 2.0).

Live Demo: backup video



THREE TERMINALS

Left: control

Middle: tcpdump, encrypted

Right: tcpdump, plaintext

A recording of the same demo: Stack-chan over WireGuard.

Video: youtu.be/kMRaY8cDx2E

AGENDA

1 **Overview & background** ~5 MIN
Project origins · NuttX & Sony · my experience

2 **The Problem & Solution** ~4 MIN
Remote access challenge · why WireGuard

3 **The Development Plan** ~2 MIN
Reuse wireguard-lwip · adapt to NuttX

4 **Version 1: Application-Based Implementation** ~4 MIN
First working tunnel · debugging challenges

5 **Version 2: The Kernel Driver** ~5 MIN
Why a driver · integration challenges · validation

6 **Live Demo** ~3 MIN
Secure remote access · Stack-chan control

7 **Summary & Reflections** ~4 MIN
Achievements · personal reflections · community

Summary

1

Secure Communication

Brought WireGuard to Apache NuttX, enabling secure IP communication with standard WireGuard peers.

2

Application to Kernel Driver

Evolved the implementation from a NuttX application into a kernel network driver, tested on real boards and QEMU.

3

Toward Upstream Contribution

Preparing the driver and related fixes for review by the Apache NuttX community.

Prepared as small, separate changes. Nothing is submitted yet.

What I learned

LESSON 1

From application user to OS contributor

For years I used NuttX from the application side.

This time I worked inside the OS: drivers and the kernel.

LESSON 2

AI helped me code, but understanding was my job

I used Claude a lot. Research and code went much faster.

OS internals were new to me. I did not take AI code as it was. I asked, read, and tested until I could explain it myself.

That was the hard part.

LESSON 3

A welcoming open-source community

Good docs and contributing guides. Active discussion and review.

Open to AI-assisted development, with people staying responsible.

CI, hackathons and events help new people join.

I hope to keep contributing to Apache NuttX and the open-source community, and I look forward to seeing NuttX continue to grow.

Acknowledgements

Alan Carvalho de Assis

For encouraging me to submit this talk to Community Over Code.

The Apache NuttX Community

For the discussions, documentation, and support throughout this project.

Sony's NuttX Developers, Past and Present

For their contributions to NuttX and the foundation they built.

The Apache Software Foundation & Community Over Code Organizers

For bringing the open-source community together, organizers and volunteers alike.

COMMUNITY OVER CODE · GLASGOW 2026

Thank You

The slides and the code are here.

wwlapaki310.github.io/gsoc2026-nuttx-wireguard/.../coc-glasgow-reveal/
SLIDES & RESOURCES

github.com/wwlapaki310/gsoc2026-nuttx-wireguard
CODE & RECORDS

SCAN TO VIEW THE SLIDES

Satoru Akita · Sony Semiconductor Solutions



Testing: What Ping Misses

SAME KEY ON EVERY BOOT?

`SPRESENSE` 3 resets, **3 different keys**.

A quick check, not a proof of good randomness.

KEYS LEFT IN MEMORY AFTER DOWN?

`SIM` session keys **all zero** after `down`.

The static key stays, on purpose, so `up` works again.

A LEAK THAT SHOWS AFTER HOURS?

`SIM` 151 rekeys, 100 down/up: **nothing lost**.

Memory back to the start-up count. Not a week-long run.

Each risk needs its own test. A passing tunnel is not one of them.

Design Choice: Trust the Clock?

Each handshake needs a newer timestamp than the peer has seen, **even after a reboot**.

Chosen

Real-time clock + the highest value used this boot.
Same rule as Linux and wireguard-go, **plus a warning** when the clock was never set.

Not chosen

Save every timestamp: wears out flash.
Reserve a range in flash: needs a write guarantee I could not verify.
Wait: as long as it last ran.

Same board, nothing sent to it: clock unset → no handshake in 75 s. Clock set → 4.1 s.

Without a clock, a device may not reconnect by itself, and a ping test will not show it.